دوره مهندسی LLM و هوش مصنوعی مولد

جلسه ۳

بازبینی، دیباگ و تست کد تولیدشده با هوش مصنوعی

موارد مرزی، APIهای ساختگی، آسیب‌پذیری‌های امنیتی و تست‌های گمراه‌کننده

کیوان محمودی

در این جلسه

۱ چرا بازبینی؟

۲ چک‌لیست بازبینی

۳ موارد مرزی و متن فارسی

۴ ساختگی و منسوخ API

۵ تزریق SQL

۶ eval و رازها در کد

۷ تست‌هایی که ابزار نوشته

۸ دیباگ با هوش مصنوعی

خطاهای رایج کد تولیدشده

نمونه	دسته
ورودی خالی، ارقام فارسی، فاصله اضافه	**موارد مرزی**
تابعی که وجود ندارد یا حذف شده	**API ساختگی یا منسوخ**
SQL Injection، کلید API در کد، eval	**امنیت**
شرط اشتباه، off-by-one، فرض پنهان	**منطق**
قابلیت و وابستگی‌ای که کسی نخواسته	**اضافه‌کاری**
تستی که رفتار غلط را تأیید می‌کند	**تست گمراه‌کننده**

کد هوش مصنوعی باورپذیر است، نه لزوماً درست. مسئولیت با برنامه‌نویس است.

چک‌لیست بازبینی

۱ فهم

- آیا هر خط را می‌فهمم؟
- ورودی مرزی چه می‌شود؟

۲ وابستگی‌ها

- همه توابع وجود دارند؟
- نسخه درست است؟
- بسته ناشناخته؟

۳ امنیت

- ورودی کاربر کجا می‌رود؟
- رازها کجا هستند؟

۴ تست‌ها

- رفتار درست را می‌سنجند
- یا فقط رفتار فعلی را؟

مطالعه موردی: شماره موبایل

```
def normalize_mobile(phone: str) -> str:
    """Normalise to 09xxxxxxxxx."""
    digits = re.sub(r"\D", "", phone)
    if digits.startswith("98"):
        digits = "0" + digits[2:]
    if len(digits) == 11:
        return digits
    raise ValueError(phone)
```

- کوتاه، تمیز و برای 09123456789 درست.
- ارقام فارسی؟ D\ در پایتون ۳ ارقام یونیکد را رقم حساب می‌کند.
- پیشوند 0098؟ شماره بدون صفر؟
- تلفن ثابت 021... هم پذیرفته می‌شود.

تست پارامتری ورودی‌های واقعی

```
@pytest.mark.parametrize("raw, expected", [
    ("09123456789",      "09123456789"),
    ("+98 912 345 6789", "09123456789"),
    ("00989123456789",   "09123456789"),
    ("9123456789",       "09123456789"),
    ("۰۹۱۲-۳۴۵-۶۷۸۹",    "09123456789"),
])
def test_valid(raw, expected):
    assert normalize_mobile(raw) == expected

@pytest.mark.parametrize("raw", [
    "02112345678", "0912345678", "", "شماره ندارم"])
def test_invalid(raw):
    with pytest.raises(ValueError):
        normalize_mobile(raw)
```

- فهرست ورودی‌های واقعی به جای حدس.
- ارزش اصلی در تست‌هاست، نه در کد.
- هر نسخه جدید را بلافاصله می‌سنجد.

موارد مرزی متن فارسی

مشکل	نمونه	مورد
کدپوینت متفاوت از 123	۱۲۳ و ۱۲۳	**ارقام فارسی و عربی**
ظاهر یکسان، رشته متفاوت	علي، کتاب	**ی و ک عربی**
split و مقایسه رشته	می‌روم و می روم	**نیم‌فاصله**
float آن را نمی‌شناسد	۱۲٫۵	**ممیز فارسی**
نامرئی ولی موجود در رشته	U+200F	**علائم جهت‌نما**

API ساختگی و منسوخ

```
# suggested by an assistant
orders = orders.append(row, ignore_index=True)
# AttributeError in pandas 2

# current API
orders = pd.concat(
    [orders, pd.DataFrame([row])],
    ignore_index=True)
```

- مدل روی کدهای قدیمی آموزش دیده: `DataFrame.append` در pandas ۲ حذف شده است.
- راستی‌آزمایی: `help`، `hasattr`، `inspect.signature` و مستندات **نسخه نصب‌شده**.
- نسخه کتابخانه‌ها را در فایل قواعد پروژه بنویسید.
- **Slopsquatting:** مهاجم بسته‌ای با نام ساختگی پیشنهادی مدل منتشر می‌کند.

تزریق SQL

```
# unsafe: user input inside the SQL text
query = f"SELECT id, name, price FROM products " \
        f"WHERE name LIKE '%{term}%'"

term = "' UNION SELECT id, name, internal_cost " \
       "FROM products --"
# -> leaks the internal cost of every product

# safe: parameterised query
db.execute(
    "SELECT id, name, price FROM products "
    "WHERE name LIKE ?", (f"%{term}%",))
```

- ورودی کاربر منطق کوئری را عوض می‌کند.
- در کوئری پارامتری مقدار هرگز SQL تفسیر نمی‌شود.
- ابزارها f-string در SQL را زیاد تولید می‌کنند.

سه الگوی خطرناک دیگر

الگو	خطر	جایگزین
کلید API در کد	لو رفتن با اولین push	متغیر محیطی یا Colab Secrets
eval روی ورودی کاربر	اجرای هر کد دلخواه	ast.literal_eval یا تجزیه صریح
shell=True با ورودی کاربر	تزریق دستور سیستم‌عامل	فهرست آرگومان‌ها

ماشین‌حساب امن با ast را در جلسه ۱۲ به‌عنوان ابزار به مدل می‌دهیم.

تست‌هایی که باگ را قفل می‌کنند

```
def apply_discount(price, percent):
    return round(price * (1 - percent / 100), -3)

# generated from the code:
assert apply_discount(500_000, 150) == -250_000  # passes!

# written from the business rules:
assert apply_discount(1_000_000, 100) == 0
with pytest.raises(ValueError):
    apply_discount(1_000_000, 150)   # fails -> bug found
```

- ابزار تست را از روی **کد فعلی** می‌نویسد، نه رفتار درست.
- `round` پایتون گرد کردن بانکی دارد؛ آیا مطلوب است؟
- چند تست را از روی **قواعد کسب‌وکار** بنویسید.

دیباگ با کمک هوش مصنوعی

- **پیام خطای کامل** همراه با traceback، نه خلاصه آن.
- کد مرتبط، ورودی‌ای که خطا ایجاد می‌کند و خروجی مورد انتظار.
- آنچه تا حالا امتحان کرده‌اید.
- درخواست کلیدی: «**اول علت را توضیح بده، بعد اصلاح کن.**»
- مراقب اصلاح سطحی باشید: `try/except` که خطا را می‌بلعد.

جمع‌بندی

- کد هوش مصنوعی باورپذیر است، نه لزوماً درست.
- تست پارامتری سریع‌ترین راه کشف موارد مرزی است.
- API ناآشنا را راستی‌آزمایی کنید؛ بسته ناشناخته نصب نکنید.
- ورودی کاربر هرگز مستقیم وارد SQL، eval یا shell نمی‌شود.
- چند تست را از روی قواعد بنویسید، نه از روی کد.

تمرین

۱ parse_price جلسه قبل را با ورودی‌های مرزی جدید محک بزنید و اصلاح کنید.

۲ apply_discount را اصلاح کنید تا تست قواعد پاس شود و گرد کردن رو به بالا باشد.

۳ کد خواندن فایل از پوشه uploads را از نظر Path Traversal بررسی کنید.

۴ برای یک تابع قدیمی خودتان تست بخواهید و تست‌های مبتنی بر قاعده اضافه کنید.